

EVOLVE

Evolving Virtual OS Learning & Verification Environment

项目团队: Or2-1

队伍编号: T2026102699911318

E 渐进式内核

V 统一虚拟环境

OS 学习文档与实践变体

VE 可信验证

项目背景：操作系统教学困境

01 原理与实践脱节

能背诵页表、调度、信号的定义，完成实践时却无法把课上所学与实践需要联系起来——“知道”与“会做”之间的鸿沟无法跨越

02 概念掌握停留在表层

学生能够背诵复述进程、虚拟内存、中断等概念，却难以解释其设计动机、运行过程和边界条件。

03 评价难以区分真实能力

一份表述规范、代码完整的作业，可能来自深入理解，也可能只是对大模型输出的整理与拼接。

04 教学复盘缺少过程证据

教师看到的只有最终提交物，难以发现学生究竟卡在概念理解、系统连接、调试方法还是因果解释等方面，教学复盘难以推进。

工程性瓶颈 · 环境侧

🔧 环境配置成本高

交叉工具链、QEMU、Rust 工具链与磁盘镜像环环相扣，任何一环版本差异都可能让实验无法启动

📁 过程证据缺失

传统验收只看“代码 + 退出码”，是否运行过、失败后如何修正一律不可见，抄袭与调试在提交物上无法区分

⚡ 反馈周期长

写完代码到得到编译错误、运行行为或评分反馈，要经过一次完整的手工构建-运行-观察循环，甚至等待助教批改

👁️ 教师核验成本高

面对全班的提交物，逐份核对“学生是否真正理解”的成本极高，最终只能退化为抽查或只看结果

项目背景：AI 浪潮的冲击

机遇 · 约束行为边界之后



一对一助教成为可能

让它解释机制、追问判断、给出分级提示，而不是替学生写代码——每个学生都负担得起、随时在场的助教，人类助教规模无法覆盖



结合真实证据回应

可以结合学生当次的编译诊断与运行记录给出针对性回应，这是传统答疑按人数分配所做不到的



分级提示引导思考

按问题线程递进的提示等级，先要求观察现象、形成假设，再给最小验证动作，最后才指向对照路径

挑战 · 放任使用之下



代写绕过思考

学生把作业整段贴给大模型，直接得到可提交的完整实现——“实验通过”不再能证明“理解发生”



幻觉冒充事实

AI 可能编造不存在的运行事实，学生把“已通过”“已验证”的幻觉当成结论写进报告



思维能力退化

提问能力本身需要被训练，逻辑思维链更需要学生自己的锻炼 AI 会严重削弱学生的思考，越用越不会问、越不会判断

我们的理解

让学生与 AI 充分合作，推动人机协同学习，让AI赋能学习而不是替代学习。

项目创新：EVOLVE

本项目未直接借鉴或复用其他项目成果，整体架构、核心功能与实验内容均由团队独立设计并自主实现。

E

Evolving

渐进式内核训练

Lab1 → Lab8

能力递归包含

“原理与实践”

V

Virtual

统一虚拟环境

QEMU virt

OpenSBI + VirtIO

“环境低成本”

OL

OS Learning

文档与实践变体

Lab 规格与脚手架

教师统一发布

“过程性教学”

VE

Verification Env.

可信证据与 AI 协作

受控断言

Agent Server

“可信可复核”

发布 → 实践 → 验证 → 辅导 → 复盘 → 评价 → 验收 → 进阶

EVOLVE: Evolving

■ ① 渐进式教学内核

一套内核通过 feature 链从最小启动逐层长出 Trap、虚拟内存、进程、文件、信号、线程——复现“操作系统如何被需求一步步逼出来”的历史脉络。学生看到的不是八份静态代码，而是同一个系统生长的全过程。

■ ② 渐进式的 lab 学习开展流程

学生始终在同一份系统上叠加机制：每个 Lab 只打开该层需要的模块，升级只补充该层文件、从不覆盖已完成的工作；任何修改都要自动接受全部前序能力的回归检验，认知负担随分层受控，成就感随能力累积。

E

Evolving
渐进式

Evolving: 渐进式 RISC-V 教学内核



PERSISTENT KERNEL BASELINE

后序能力复用 Trap、页表、进程与 fd；每次变更都接受 Lab1–Lab(n-1) 回归。

Evolving: 可测机制与内核策略分离

API

L4 · USER / ABI

用户程序 · syscall ABI · 教学测试

K

L3 · KERNEL INTEGRATION

feature 选择 · Trap 分派 · 调度策略 · 设备集成 · Lab 回归

8x

L2 · COMPONENT CRATES

os-sbi

os-context

os-syscall

os-alloc

os-vm

os-fs

os-signal

os-sync

T

L1 · TEST SURFACES

Host 单元测试: 纯逻辑快速回归

QEMU Recipe: 硬件与调度可信验证

EVOLVE: Virtual

① 环境配置

全部实验运行于 QEMU `virt` 虚拟机：OpenSBI 固件、Sv39 分页、VirtIO 块设备，全链路 RISC-V 64。交叉工具链、QEMU 版本、磁盘镜像的配置差异被整体收编，“首周耗在环境搭建”的传统开销不复存在。

② 零差异 · 可重复 · 可回滚

零差异——所有人面对同一组硬件事实；可重复——受控 Runner 以子进程驱动 QEMU 并捕获串口输出，同一份代码随时重演同一行为；可回滚——工作区快照支持整份恢复，实验永远可以从已知状态重新开始。



Virtual
虚拟环境

Virtual: 统一虚拟环境配置



ISA / MMU

RISC-V 64 · Sv39

BLOCK DEVICE

VirtIO MMIO 0x1000_1000

RECOVERY UNIT

固定镜像 + Recipe

零差异

可重复

可回滚

EVOLVE: OS Learning

OL

OS Learning
操作系统学习

■ ① 教学编排的实验文档

8 册 Lab 手册按“机制为何被逼出”的动机线组织；答案与正文分离、不进学生检索；每 Lab 附概念目录与误区说明；lab.yaml 机器可读规格让手册、脚手架、Prompt、验证命令共享一份契约。

■ ② 实践变体

fill 补全代码 / debug 定位错误 / remedial 补救练习三种形态；教师下发 random 时，服务端为每名学生写死一个具体变体；负向断言保证变体基线“应失败”——整段抄来的代码无法通过。

■ ③ 以学生学习为中心

手册、脚手架、诊断、提示、复盘，全部指向同一个目标——推动学生“形成自己的判断”，而不是更快地拿到一份能跑的代码。

OS Learning: 教学编排与设计



LAB FACTORY



TEACHER CONTROL



OS Learning: 统一学生工作台

● ● ●

EVOLVE WORKBENCH

workspace: student/42/lab6 · runId: r-240816-1732

01 · 读手册

Lab6 · VirtIO 文件系统

目标
从内嵌文件走向块设备

检查点
MMIO 映射
fs.img 挂载
文件断言

Scaffold 只加入
当前 Lab 新文件

02 · 改代码 / Monaco

kernel/src/fs/virtio_block.rs

41 pub fn read_block(block_id: usize, buf: &mut [u8]) {

42 let sector = block_id * SECTORS_PER_BLOCK;

43 device.read_sector(sector, buf)?;

44 }

当前文件 hash 与 Scaffold 基线持续比较

03 · 跑 Recipe / Terminal

\$ make test-lab6

PASS fs_create · PASS mmap · PASS spawn

TRACE_V1 qemu_exit=success · assertions=12/12

04 · 查 Problems

E0308 · 类型不匹配

virtio_block.rs:43:18

点击回跳源码位置

05 · 问 Tutor

“先比较 expected 与 observed。
你的 MMIO 映射发生在页表激活前
还是之后？”

refs: run:... · diag:... · kb:...

06 · 复验 & 报告 & 评分

基于实验过程中行为进行复盘提问
基于预设报告模板进行填写

12

OS Learning: 统一学生工作台

EVOLVE

首页 入门指南 引导式学习 学习材料

系统构建路径

手册 工作区 学习支持

test

实验手册

实验 1: 裸机启动与最小内核

相关教材理论:

[第 2 章 · 操作系统介绍 \(P16\)](#)

[第 6 章 · 机制: 受限直接执行 \(P49\) 开头引言部分](#)

AI 导师

gpt-5.5

AI 导师

从机器上电到内核打印第一行文字, 中间至少经过哪些环节? 先写下你的判断, 我们再沿入口和链接地址逐层验证。

先说我的判断 整理失败现象 寻找原因证据 整理善解证据

描述你观察到的现象或卡住的地方...

Ctrl + Enter 发送

一、问题场景

OSTEP 从虚拟化、并发和持久性三条主线展开操作系统原理, 但这些内容都隐含了一个基本前提: 内核已经被加载到内存, 并且正在正常运行。

工作区 代码编辑与运行共用当前账号目录

目录

kernel/build.rs

build.rs

文件

我的系统

kernel

src

build.rs

Cargo.toml

终端

测试结果

```
$ cargo run -p kernel --features lab1 --release
```

终端 1

学习支持

实验报告

学习评价

Lab1 实验报告

上次保存: 2026/8/17 12:07:06 · 当前: Markdown 文档

AI 点评 导出报告 提交给老师 保存

编辑 分栏 预览 H₂ B I </> 图片 附件 导入

正文 170 字符 最终排版 提交内容

一、实验目标与准备

Lab1 裸机启动与 SBI · 实验报告

EVOLVE: Verification Environment

■ ① 可信验证、受控断言

verified = trusted + 退出码 0 + 断言非空且全部通过；运行不经 shell、只允许白名单程序，行为断言、Trace 机器帧与 SHA-256 制品哈希把"运行过"提升为"可信验证通过"。

■ ② Agent Server 协作学习

Tutor 引导办学，拒绝直接给出答案，而是引导学生深入思考
Assessment 依证据评分，双 Agent 协同完成苏格拉底复盘。

■ ③ 证据即边界

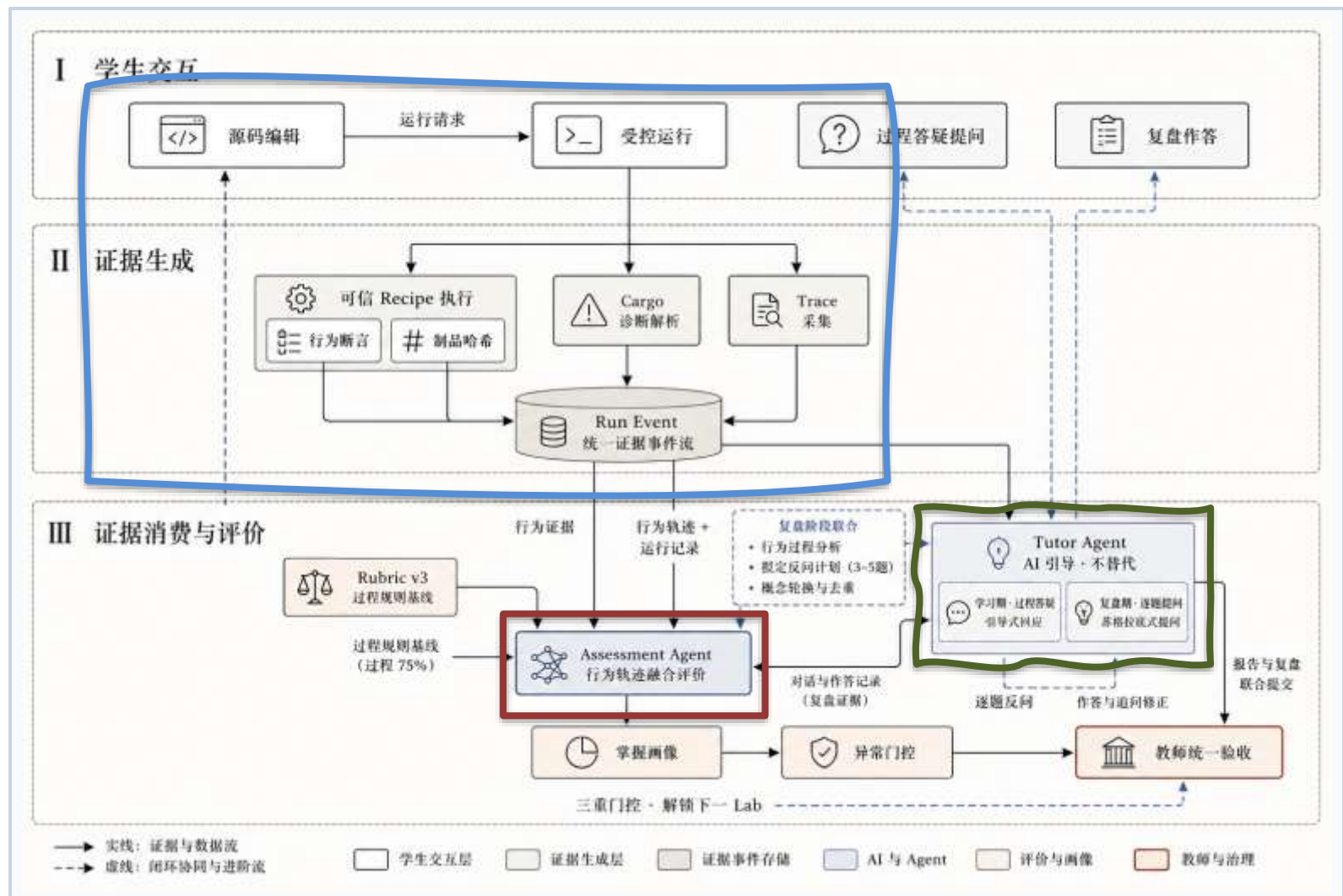
模型不得声称执行过不存在的运行；引用必须命中证据白名单；口头解释永远不能覆盖可信运行——这是 AI 协作学习的安全底座。

VE

Verification
Environment

可信验证环境

Verification Env.: 证据与数据流



可信验证

objective · passCriteria · evidenceRefs

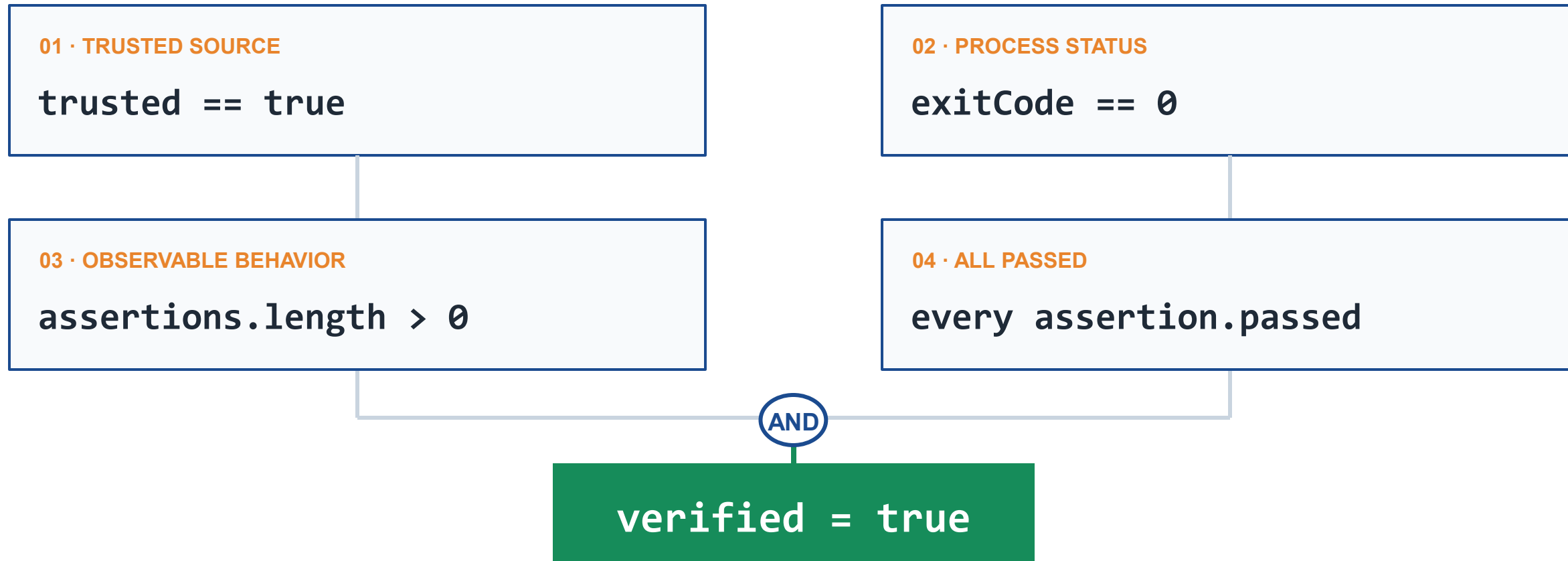
Tutor Agent

concepts · code · logic

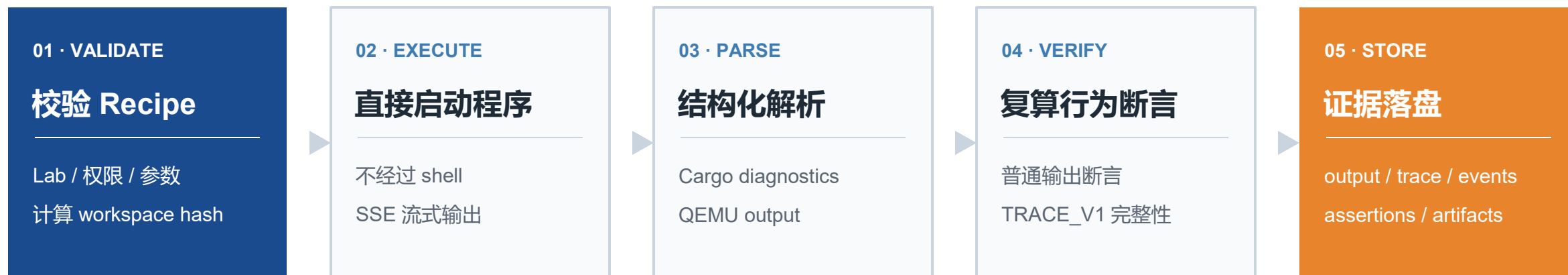
Assessment Agent

behavioral evidence · dialogue history

Verification Env.: 可信验证



Verification Env.: 过程行为证据



ONE EVIDENCE LEDGER

`runId · userId · labId · workspaceHash · trusted · verified · diagnostics · events · artifacts`

同一份结果被三个角色消费

Tutor · 解释与追问

Assessment · 独立评价

教师 · 复核与反馈

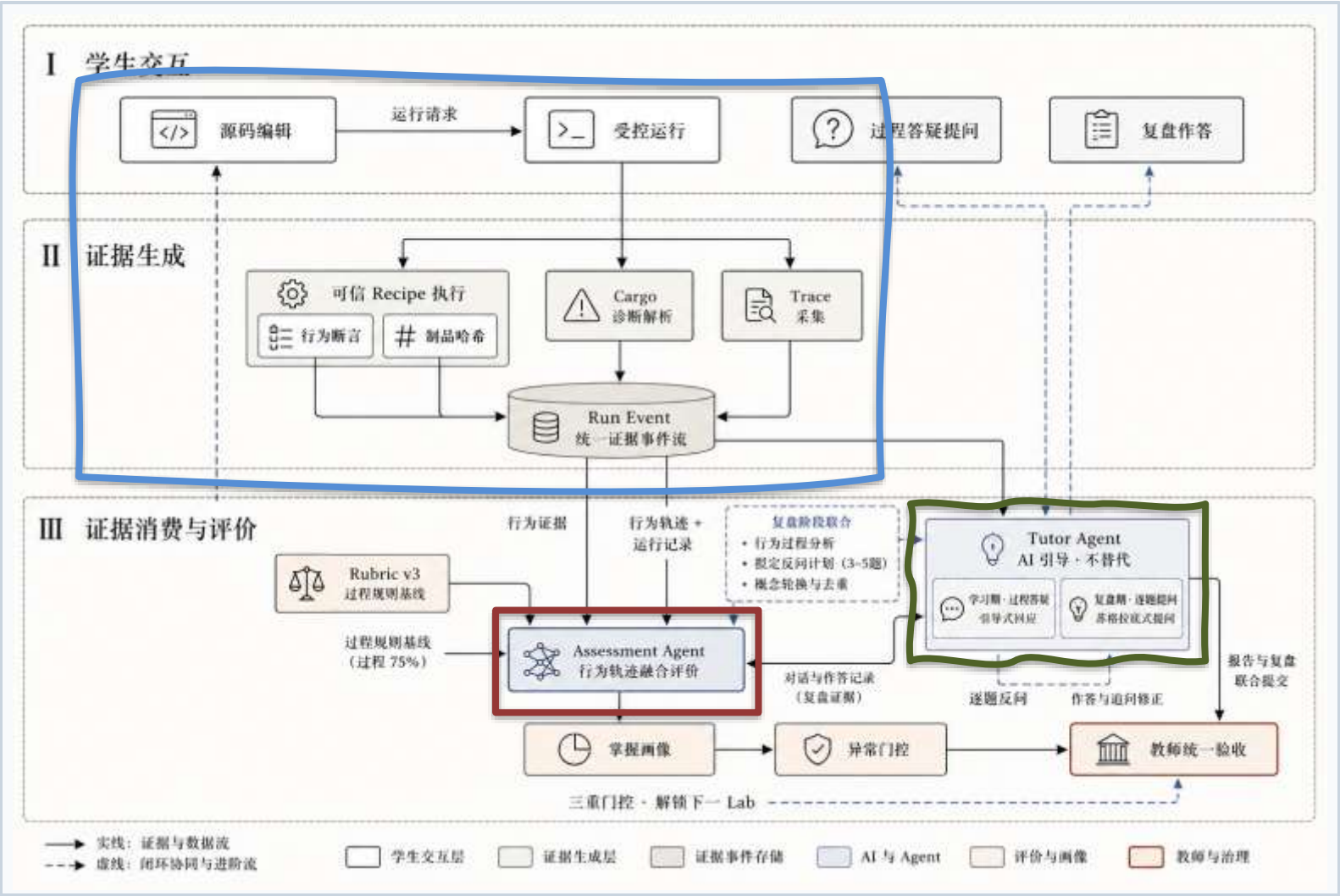
run: 运行

trace: 轨迹

diag: 诊断

kb: 知识引用

Verification Env.: 证据与数据流



可信验证

objective · passCriteria · evidenceRefs

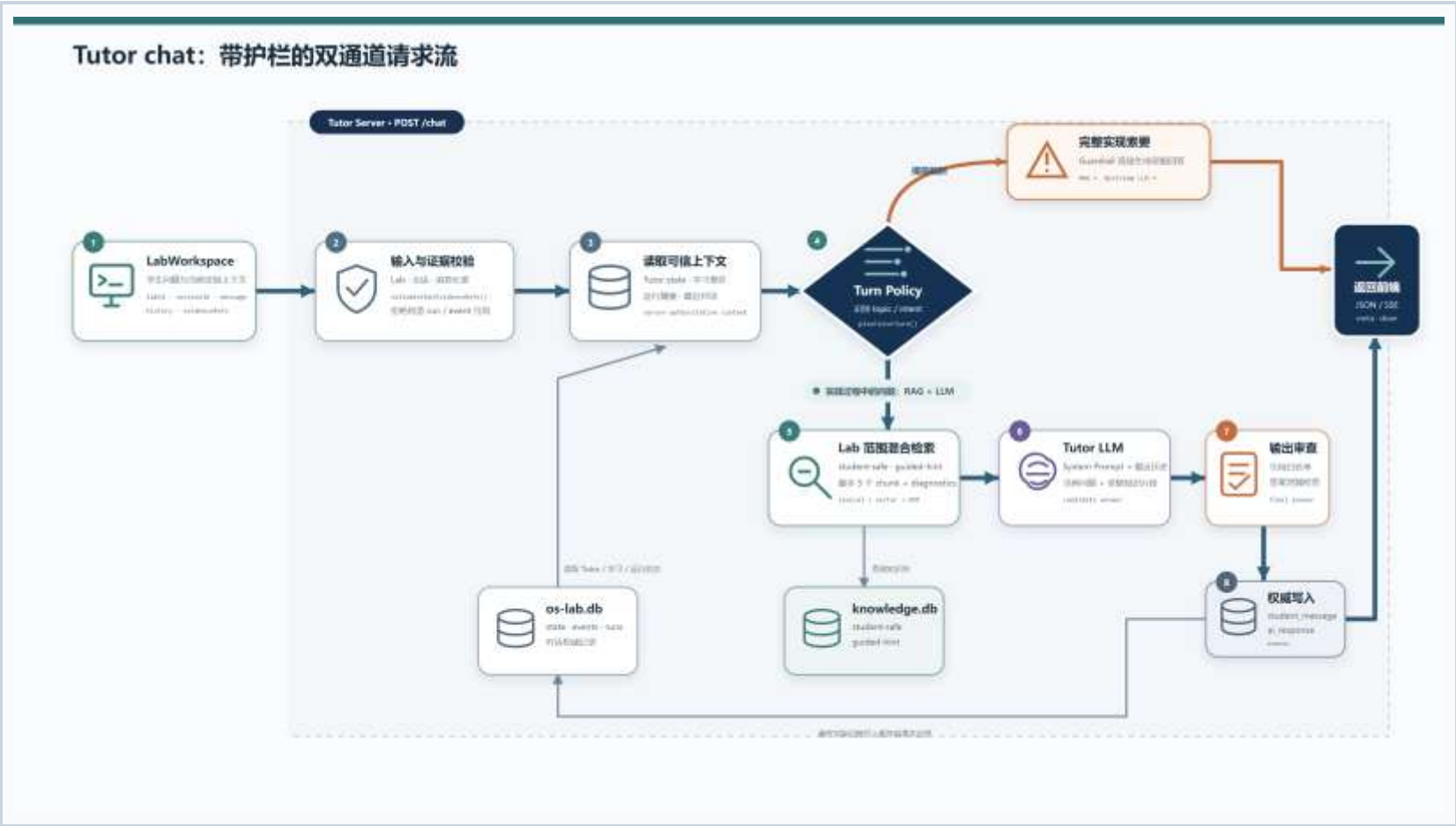
Tutor Agent

concepts · code · logic

Assessment Agent

behavioral evidence · dialogue history

Verification Env.: Tutor Agent



Harness

1 · INTENT

7 类意图：概念、读码、调试、验证、反思、迁移、直接答案

2 · GUARDRAIL

完整代码 / 最终答案 → 立即受限引导
不检索，不发送可拼装答案片段

3 · LIMITED CONTEXT

4 种回应模式独立于意图
每个 topic 独立递进 L0-L4

4 · POST CHECK

证据白名单 · 引用白名单
禁止虚构 “已运行 / 已通过”

Verification Env.: Assessment Agent



Harness

1 · AUTHORITY EVIDENCE

结构化事件 + 可信运行
events / runs / transcript / evidenceRefs

2 · RULE BASELINE

规则基线Rubric v3.3.0
过程 75% · 复盘 25%

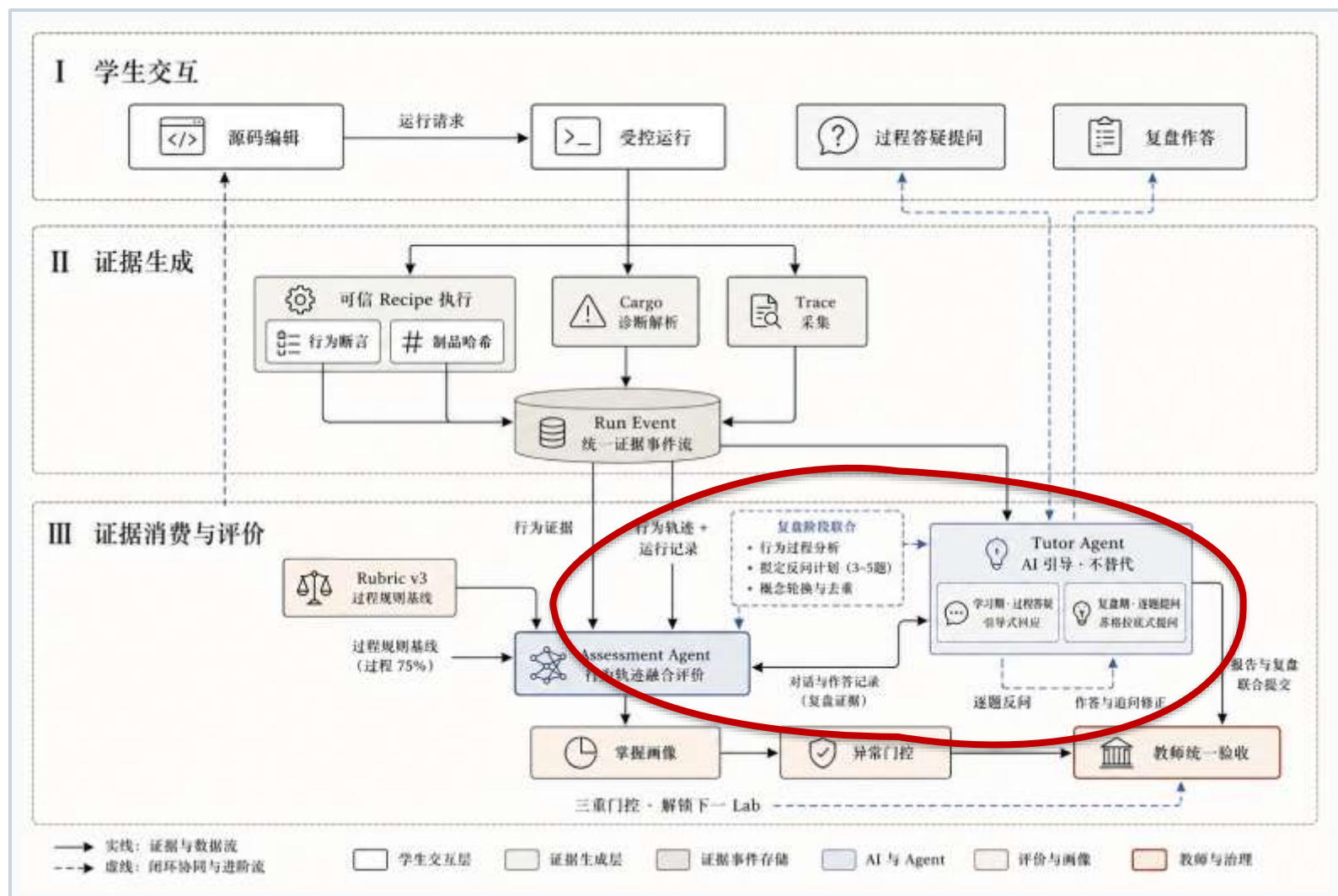
3 · CONSTRAINED AGENT

独立评分, 证据受限
不能修改 verified 事实

4 · FUSION & PERSISTENCE

规则 60% · Agent 40%
复盘 · 掌握度 · 教师验收

Verification Env.: 证据驱动的苏格拉底式复盘



01 · ASSESSMENT

定义复盘目标与证据门槛

02 · TUTOR

参考 brief 进行苏格拉底复盘

03 · ASSESSMENT

结合行为进行学习评价

TEACHER IN CONTROL

教师参考与反馈

系统测试情况

AUTOMATED / CONTRACT TESTS

215/215

全部通过

143 Node

40 Rust

32 Python

覆盖 Handbook / Tutor / Learning 与 8 个组件 crate

QEMU END-TO-END

8/8 Labs

行为断言全部通过

TUTOR HARNESS

19 泄漏率 0

相关性、引导、引用均达标

ASSESSMENT HARNESS

5 用例通过

计划、判定、反馈契约

KNOWLEDGE BASE

1,384 active

1,360 indexed chunks



三方环境框架对比

比较维度	EVOLVE	tg-rcore	xv6
代码组织与演进	单内核 feature 链 演进本身可切换、可观察	分章工程 + 共享 crate 章节边界表达演进	单一成熟源码树 增量依赖指导与版本对照
知识分布与教学节奏	削峰铺轨 先主线、后完整语义	章内做透 标准化练习，峰值更陡	完整系统在场 适合概念后的深化
实践形态与验收	Web 手册 + 隔离工作区 + 教师端 QEMU 断言 / make test-labN + 40 项 host 单测	本地 clone + 终端为主 tg-rcore-tutorial-checker 判题	教材 + 终端 lab，教务自建 make qemu + grade-lab-* / usertests
AI 时代的教学应对	证据驱动Agent、教师控制内建 学习活动可见、内置 AI设置答案护栏 报告复盘可复核	经典内容与判题强 过程无监督，AI无限制	成熟内容与评分脚本强 过程无监督，AI无限制

项目意义

🖥️ 对学生：AI 助教与思维引导

- 一对一、全程在线、行为可控的 AI 助教 + 零配置实验环境
- "提出判断—设计验证—观察—修改复验"的工程师思维训练
- 复盘机制引导学生学会自我思考与反思

📦 对课程教学：可演进可迭代的教学内核

- 内核以 Rust 组件 crate 形态组织，每个组件可独立单元测试
- 具备发布到 crates.io 的工程条件，回应赛题组件化要求
- 教学容量显式声明，为课程复用与教学演进提供清晰接口

👁️ 对教师：过程可见与教学反馈

- 黑箱提交物变为附带完整证据链的过程记录
- 统一验收界面保留教师最终判断权与审计记录
- Agent Server辅助评估，降低核验成本

🔗 对技术范式：可迁移可落地的融合路径

- LLM + 可信证据 + 制度化评价的融合路径尝试
- 灵活迁移到其他"过程可信 + AI 辅导"的实践教学场景

开发迭代核心路径

方向一 · 内核能力

Lab5 → Lab8: VirtIO 磁盘、信号、线程与死锁检测

方向二 · 教学平台

静态手册 → 双端教学/学习区、账号、可信运行与报告

方向三 · 智能教学

Tutor / Assessment Agent 与证据约束的融合评价



每阶段修改以**机器可读契约**为接口边界，以**测试 fixture 与真实 HTTP/SQLite Smoke**收口验收；新增功能同步更新测试与文档，防止手册、Scaffold、Prompt 与验证命令各自漂移。



分工与协作

AM

AM-SuSh

总体集成与服务端证据链

SZ

SIZN

渐进式内核与IDE交互

RT

RnTs1002

教学编排与实验任务设计

AI 协作说明

工具	用途
Cursor IDE等	文件编辑、代码解释、验证命令编排等
ChatGPT等	概念知识讨论、debug、文档措辞参考等

更多详细说明请见实验文档

非本队来源说明

来源	用途
tg-rcore-tutorial	赛题 30% 参考练习基线
MIT 6.S081 xv6-riscv	教学环境框架对比
OSTEP、CSAPP、 RISC-V Reader	实验手册背景和受限 RAG 知识来源
OpenSBI、QEMU、 Rust、Cargo crates	编译、启动、硬件模拟和运行依赖
外部LLM服务	支持Agent Server实现

未来优化方向

1. 深化内核机制

逐步补充 COW、文件映射、持久化元数据和崩溃一致性，使各实验模块从相对独立的功能验证进一步形成完整、连续的系统语义。
完善资源耗尽时的错误处理与回收机制；扩展到更通用的对象生命周期和多核调度，更好实现和推动学生的个性化学习。

2. 扩展验证与落地观测

后续验证应从关键事件扩展到覆盖内存、进程、文件系统和并发机制的统一观测体系，并加强跨模块行为关联。
开展匿名化的多班级真人对照实验，预注册指标并报告样本量、效应量和失败案例，进一步完善工程缺陷并持续观测落地效果。

3. 提升平台部署与执行安全

演进为标准化、隔离化和可扩展的运行环境，完善计算资源限制、、运行监控与异常恢复等，满足公开部署和多班级并发使用的需要。
完善统一开发环境、服务部署、健康检查和数据恢复方案，进一步整个功能到教师工作台。

EVOLVE

Evolving Virtual OS Learning & Verification Environment

Evolving

渐进式内核

Virtual

虚拟环境

OS Learning

操作系统实验教学

Verification Env.

可信证据与 AI 协作